

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

APLICAȚIE ANDROID PENTRU GĂSIREA UNOR LOCAȚII PERSONALIZATE ÎNTR-UN ORAȘ

Absolvent

Andrei Oprescu

Coordonator științific

Conf. dr. Radu Boriga

București, iunie 2026

Rezumat

Această lucrare prezintă proiectarea și implementarea aplicației FindOut, o aplicație mobilă pentru Android care ajută utilizatorii să descopere locații dintr-un oraș în funcție de preferințele lor personale. Platformele actuale de recenzii afișează, în general, aceeași listă de locații tuturor utilizatorilor, ordonată după un punctaj general, fără a ține cont de ceea ce contează pentru fiecare persoană în parte. Pentru a răspunde acestei probleme, aplicația permite utilizatorului să stabilească importanța pe care o acordă mai multor concepte (precum atmosfera, curățenia sau prețul), iar pe baza acestora calculează un scor de potrivire între locație și utilizator. Pe lângă recomandările personalizate, aplicația oferă o funcționalitate de descoperire a unor locații pe care utilizatorul nu le-ar alege în mod obișnuit, precum și o căutare tolerantă la greșeli de tastare, adaptată limbii române. Sistemul este alcătuit dintr-un client Android nativ, scris în Kotlin cu Jetpack Compose, un server REST realizat cu Spring Boot și o bază de date PostgreSQL cu extensia PostGIS pentru datele geospațiale. Rezultatul este o aplicație funcțională care oferă recomandări personalizate și încurajează descoperirea de locații noi, mai puțin dependente de popularitatea acestora.

Cuvinte-cheie: aplicație Android, Jetpack Compose, sistem de recomandare, PostGIS, căutare tolerantă la greșeli (fuzzy), Spring Boot.

Abstract

This thesis presents the design and implementation of FindOut, a mobile Android application that helps users discover places in a city according to their personal preferences. Current review platforms generally show the same list of places to all users, ordered by an overall rating, without taking into account what matters to each person individually. To address this problem, the application lets the user set the importance they give to several concepts (such as atmosphere, cleanliness or price) and, based on these, computes a match score between a place and the user. Besides personalized recommendations, the application provides a feature for discovering places the user would not normally choose, as well as a typo-tolerant search adapted to the Romanian language. The system consists of a native Android client, written in Kotlin with Jetpack Compose, a REST server built with Spring Boot, and a PostgreSQL database with the PostGIS extension for geospatial data. The result is a working application that provides personalized recommendations and encourages the discovery of new places, less dependent on their popularity.

Keywords: Android application, Jetpack Compose, recommendation system, PostGIS, fuzzy search, Spring Boot.

Cuprins

1 Introducere **5**

1.1 Contextul și problema 5

1.2 Soluția și utilitatea aplicației 6

1.2.1 Soluția 6

1.2.2 Utilitatea aplicației 6

1.2.3 Ușurința în utilizare 7

1.3 Contribuții și structura lucrării 7

2 Preliminarii **8**

2.1 Noțiuni fundamentale 8

2.1.1 Arhitectura client-server RESTful 8

2.1.2 Modelul Model-View-ViewModel 8

2.1.3 Sisteme de recomandare 9

2.1.4 Potrivirea aproximativă a șirurilor de caractere 9

2.1.5 Date geospațiale și indexarea spațială 9

2.1.6 Autentificarea pe bază de token și stocarea parolelor 9

2.2 Stadiul actual și soluții existente 10

2.3 Obiectivele lucrării 10

3 Tehnologiile utilizate **11**

3.1 Aplicație nativă Android 12

3.2 Interfața declarativă cu Jetpack Compose 12

3.3 Serverul cu Spring Boot 12

3.4 Autentificarea cu token-uri JWT 12

3.5 Sursa datelor: OpenStreetMap și Overpass Turbo 13

3.6 Google Places API 13

4 Proiectarea și implementarea sistemului **14**

4.1 Arhitectura aplicației 14

4.2 Backend 15

4.3 Client MVVM 16

4.4 Baza de date 18
4.5 Algoritmii folosiți 19
4.5.1 Scorurile medii ale atributelor 19
4.5.2 Scorul de potrivire 20
4.5.3 Scorul de descoperire 20
4.5.4 Căutarea tolerantă la greșeli și clasarea rezultatelor 21

5 Prezentarea aplicației 21
5.1 Pornirea aplicației și autentificarea 22
5.2 Stabilirea preferințelor 22
5.3 Ecranul principal și descoperirea locațiilor 23
5.4 Căutarea locațiilor 24
5.5 Pagina unei locații 24
5.6 Scrierea unei recenzii 25
5.7 Salvarea locațiilor și profilul 26

6 Concluzii 27

Bibliografie 28

Capitolul 1

Introducere

Această lucrare de licență se încadrează în subdomeniul dezvoltării de aplicații mobile și folosește elemente din domeniul sistemelor de recomandare și al datelor geospațiale. Tema o reprezintă proiectarea și implementarea unei aplicații Android, numită FindOut, prin care utilizatorii pot descoperi locații dintr-un oraș în funcție de preferințele lor personale.

1.1 Contextul și problema

Atunci când explorează un oraș, un vizitator își dorește de cele mai multe ori o experiență cât mai completă, atât din punct de vedere cultural, cât și recreațional. Acest echilibru este însă unul personal și diferă de la o persoană la alta, în funcție de preferințele fiecăruia. În linii mari, și fără a fi o împărțire exhaustivă, locațiile care pot interesa un vizitator se pot grupa în două categorii, după tipul de activitate pe care îl presupun: activități turistice, legate de vizitarea atracțiilor, a muzeelor sau a locurilor cu însemnătate istorică ori culturală, și activități recreaționale, desfășurate în restaurante, baruri, parcuri, terenuri sportive, cluburi, parcuri de distracții și altele. Pentru ambele categorii există mai mulți factori care influențează percepția vizitatorului asupra experienței și care țin strict de gusturile personale.

Problema cu care se confruntă un turist nu ține de lipsa informației, ci de natura acesteia. Deși platforme precum Google Maps sau TripAdvisor, alături de bloguri și site-uri de specialitate, oferă date despre aproape orice punct de interes dintr-un oraș, aceste informații sunt în mare parte lipsite de nuanță și de personalizare. Recomandările iau frecvent forma unor liste standardizate (de tipul ”Top 10 restaurante”), astfel încât utilizatori cu preferințe diferite primesc, în esență, aceleași 10-15 sugestii. Chiar și atunci când destinațiile propuse sunt printre cele mai bine cotate, clasamentele nu garantează relevanța pentru gusturile individuale și pot fi influențate de promovarea plătită.

La nivel de ansamblu, acest mecanism generează un efect cumulativ: într-un oraș cu sute de localuri aflate în continuă schimbare, direcționarea constantă a turiștilor către aceleași destinații populare defavorizează locațiile noi sau mai puțin cunoscute. Pe termen lung, acest dezechilibru

afectează economia locală și turismul, cu un impact resimțit îndeosebi de micile afaceri.

1.2 Soluția și utilitatea aplicației

1.2.1 Soluția

FindOut este o aplicație care ajută turiștii și, în general, persoanele dornice de experiențe noi, dar familiare, să descopere locații recomandate în funcție de preferințele lor individuale, cu ajutorul unui algoritm de potrivire și al unui set de atribute detaliate. După autentificare, un utilizator își stabilește propriile preferințe, acordând un scor de importanță de la 0 la 5 fiecăruia dintre cele 23 de concepte. Fiecare concept se regăsește în una sau mai multe dintre categoriile de locații din aplicație, iar scorul reflectă cât de mult contează acel aspect pentru utilizator atunci când alege o locație.

Fiecărei locații îi este asociat un set de atribute specifice categoriei din care face parte (de exemplu, pentru o cafenea: calitatea cafelei, ambientul, decorul, viteza servirii). Fiecare are un scor mediu, între 0,5 și 5 stele, calculat pe baza punctajelor primite în recenziile celorlalți utilizatori. Folosind aceste scoruri împreună cu preferințele utilizatorului, algoritmul calculează un scor de potrivire între locație și utilizator, exprimat pe o scară de la 0 la 100%. Pe baza tuturor scorurilor medii ale atributelor unei locații se calculează și punctajul general al acesteia, echivalentul unui rating de pe Google Maps. Pentru a putea lăsa o recenzie, un utilizator trebuie să acorde un scor pentru cel puțin trei atribute ale locației, scorul total al recenziei fiind media acestor atribute.

Pe lângă funcționalitatea principală de recomandare, aplicația oferă mai multe opțiuni suplimentare. Una dintre ele este salvarea unei locații pentru a fi accesată mai târziu. O alta este descoperirea de experiențe noi, care folosește o variantă a algoritmului principal: caută locațiile cu atributele cele mai bine cotate, dar care corespund conceptelor cu importanța cea mai mică din preferințele utilizatorului. Astfel, sunt recomandate locații care în mod normal nu s-ar potrivi gusturilor sale, cu scopul de a-l încuraja să încerce ceva nou; dacă nu este mulțumit de locația propusă, poate genera o altă recomandare de acest tip. Nu în ultimul rând, există și o căutare manuală a locațiilor, realizată cu ajutorul unui algoritm de potrivire tolerantă la greșeli (fuzzy), care afișează sugestii pe măsură ce utilizatorul tastează.

1.2.2 Utilitatea aplicației

Elementul care diferențiază aplicația este recomandarea personalizată: combinarea mai multor atribute, fiecare cu punctaj individual, împreună cu preferințele setate de utilizator, oferă un nivel de personalizare pe care platformele bazate pe scoruri generale, precum Google Maps, nu îl au [12]. Un exemplu simplu: un restaurant poate avea un rating de 4,0 stele pe Google, însă acea diferență de 1,0 se poate datora lipsei de curățenie, un aspect care nu i-a deranjat suficient

pe cei care au lăsat recenzii cât să îl menționeze. Pentru un alt utilizator, aflat în căutarea unei locații noi, curățenia ar putea fi un factor decisiv, iar acesta ar putea fi neplăcut surprins în timpul vizitei. Aplicația surprinde tocmai astfel de aspecte, prin scorurile acordate pe fiecare atribut în parte.

În plus, algoritmul nu pune accent pe popularitatea unei locații, astfel încât sunt recomandate și locuri mai puțin cunoscute. Acest lucru aduce un element de descoperire și sprijină indirect locațiile mai noi sau mai mici, care altfel ar fi greu de găsit.

1.2.3 Ușurința în utilizare

Aplicația este simplu de folosit și presupune un proces minim de inițiere [6]. Practic, utilizatorul trebuie doar să își stabilească preferințele, adică să indice cât de important este pentru el fiecare concept atunci când caută o locație potrivită. Imediat după crearea contului, serverul folosește aceste preferințe împreună cu poziția dispozitivului și, pe baza datelor pe care le deține deja despre locațiile din apropiere, calculează și afișează locațiile recomandate, fiecare însoțită de scorul său de potrivire. În acest fel, valoarea aplicației este vizibilă încă de la prima utilizare, fără configurări complicate.

1.3 Contribuții și structura lucrării

Principalele contribuții ale acestei lucrări sunt proiectarea și implementarea unui sistem complet, de tip client-server, format dintr-o aplicație mobilă nativă pentru Android și un server REST realizat în Spring Boot, împreună cu o bază de date PostgreSQL cu extensii pentru date geospațiale. Pe partea de logică, am dezvoltat un algoritm de recomandare personalizată, bazat pe scorurile pe atribute și pe preferințele utilizatorului, precum și un mecanism complementar prin care sunt scoase în evidență locații mai puțin cunoscute. De asemenea, am implementat o căutare tolerantă la greșeli de tastare și filtrarea locațiilor în funcție de poziția geografică a utilizatorului.

Lucrarea este structurată în patru capitole de conținut, precedate de introducere și urmate de un capitol de concluzii. Capitolul Preliminarii prezintă noțiunile teoretice pe care se sprijină tema, situează aplicația în raport cu soluțiile deja existente și formulează obiectivele lucrării. Capitolul Tehnologiile utilizate descrie tehnologiile folosite, motivează alegerea lor și le compară cu alternative relevante, pentru a justifica deciziile legate de arhitectură. Capitolul Proiectarea și implementarea sistemului prezintă arhitectura pe niveluri, modelul de date și diagramele aferente (printre care diagrama entitate-relație), precum și algoritmii folosiți. Capitolul Prezentarea aplicației conține prezentarea funcționalităților aplicației din perspectiva utilizatorului, însoțite de capturi de ecran. În final, capitolul de concluzii rezumă rezultatele obținute, limitările lucrării și posibilele direcții de dezvoltare ulterioară.

Capitolul 2

Preliminarii

Acest capitol prezintă noțiunile teoretice și tehnice pe care se bazează aplicația, situează lucrarea în raport cu soluțiile deja existente și formulează, pe baza acestui context, obiectivele propuse.

2.1 Noțiuni fundamentale

2.1.1 Arhitectura client-server RESTful

Aplicația este construită după modelul client-server: un client, instalat pe dispozitivul utilizatorului, trimite cereri către un server care deține logica și datele și care întoarce răspunsuri. Comunicarea dintre cele două respectă stilul arhitectural REST (Representational State Transfer), introdus de Fielding [2]. Într-o arhitectură REST, resursele (în cazul de față locații, recenzii, preferințe) sunt identificate prin adrese, iar operațiile asupra lor sunt exprimate prin verbele protocolului HTTP. O proprietate importantă a acestui stil este faptul că este lipsit de stare (stateless): fiecare cerere conține toate informațiile necesare procesării sale, serverul nepăstrând o stare a conversației între cereri. Această proprietate simplifică serverul și îi permite să fie scalat mai ușor.

2.1.2 Modelul Model-View-ViewModel

Pentru organizarea codului aplicației de pe dispozitiv se folosește modelul arhitectural Model-View-ViewModel (MVVM), recomandat de documentația oficială Android [1]. În acest model, interfața (View) este separată de logica de prezentare (ViewModel) și de date (Model). ViewModel-ul expune starea de care are nevoie interfața și o actualizează, fără a cunoaște detalii despre modul în care aceasta este afișată. Avantajul principal este separarea responsabilităților: interfața se ocupă doar de afișare, iar starea și prelucrarea ei sunt izolate, ceea ce face codul mai ușor de înțeles și de testat.

2.1.3 Sisteme de recomandare

Un sistem de recomandare are rolul de a selecta, dintr-o mulțime mare de elemente, pe cele mai potrivite pentru un anumit utilizator [12]. Spre deosebire de o listă generală, identică pentru toți, o recomandare ține cont de preferințele individuale. Abordarea folosită în această lucrare este una bazată pe conținut: locațiile sunt descrise printr-un set de atribute, utilizatorul își exprimă preferințele față de aceste atribute, iar potrivirea dintre cele două este calculată printr-un scor [7]. O preocupare cunoscută a sistemelor de recomandare este echilibrul dintre exploatare, adică oferirea de elemente despre care se știe deja că plac utilizatorului, și explorare, adică propunerea unor elemente noi; lucrarea de față tratează acest echilibru printr-o funcționalitate dedicată de descoperire.

2.1.4 Potrivirea aproximativă a șirurilor de caractere

Căutarea după nume trebuie să funcționeze și atunci când utilizatorul scrie greșit. Acest lucru se bazează pe potrivirea aproximativă a șirurilor, un domeniu care studiază măsurarea similarității dintre două șiruri de caractere [8]. O metodă răspândită folosește trigramele: orice șir este descompus în grupuri de câte trei caractere consecutive, iar gradul de similaritate este dat de proporția de trigrame comune [14]. Deoarece o greșeală de tastare schimbă doar câteva trigrame, scorul de similaritate rămâne ridicat, ceea ce permite regăsirea cuvântului corect.

2.1.5 Date geospațiale și indexarea spațială

O parte importantă a aplicației o reprezintă filtrarea locațiilor după distanța față de poziția utilizatorului. Datele geospațiale descriu poziții pe suprafața Pământului, exprimate de regulă prin coordonate de latitudine și longitudine. Calculul corect al distanțelor dintre astfel de coordonate nu este simplu, deoarece Pământul nu este plan. Sistemele specializate stochează poziția ca pe o valoare spațială distinctă și oferă funcții dedicate pentru calculul distanțelor. Pentru ca aceste calcule să fie eficiente atunci când există multe locații, se folosesc indecși spațiali [3], structuri care permit găsirea rapidă a punctelor dintr-o anumită zonă, fără parcurgerea întregii mulțimi de date.

2.1.6 Autentificarea pe bază de token și stocarea parolelor

Pentru ca serverul să știe ce utilizator face o cerere, fără a păstra o stare de sesiune, se folosește autentificarea pe bază de token. La autentificare, serverul emite un token de tip JSON Web Token (JWT), un token semnat care conține identitatea utilizatorului [5]. Clientul atașează acest token la fiecare cerere, iar serverul îi verifică semnătura pentru a avea încredere în el, fără a reține nimic între cereri. În ceea ce privește parolele, acestea nu sunt niciodată stocate ca atare; se stochează doar o valoare obținută printr-o funcție de tip hash. Pentru aceasta se folosește

algoritmul bcrypt, conceput pentru a fi intenționat lent și pentru a permite creșterea în timp a costului de calcul, ceea ce face atacurile prin încercări repetate mult mai dificile [11].

2.2 Stadiul actual și soluții existente

Pe piață există mai multe aplicații care oferă informații despre locații dintr-un oraș. Cele mai cunoscute sunt aplicațiile de hărți și de recenzii, precum Google Maps și TripAdvisor, la care se adaugă numeroase bloguri și site-uri de specialitate ce publică liste de tipul ”cele mai bune locații dintr-un oraș”. Aceste platforme oferă un volum mare de informații și recenzii, însă prezintă câteva limitări din perspectiva temei de față.

În primul rând, ele afișează în general aceeași listă, ordonată după un punctaj general, tuturor utilizatorilor. Acest punctaj nu ține cont de ceea ce contează pentru o anumită persoană: un utilizator poate acorda importanță liniștii și curățeniei, altul prețului sau atmosferei, însă lista rămâne aceeași. În al doilea rând, recenzia se reduce de obicei la un singur scor global, care amestecă toate aspectele unei locații; un punctaj de patru stele nu spune dacă locația a pierdut o stea din cauza curățeniei sau a prețului, deși această distincție poate fi decisivă pentru un alt vizitator. În al treilea rând, ordonarea după popularitate tinde să direcționeze utilizatorii mereu către aceleași locații deja cunoscute, dezavantajând locațiile mai noi sau mai puțin populare.

Față de aceste soluții, aplicația propusă urmărește o recomandare personalizată, bazată pe preferințele individuale exprimate pe mai multe dimensiuni, recenzii detaliate la nivel de atribut și o componentă de descoperire care nu se bazează pe popularitate. Astfel, lucrarea se încadrează în zona aplicațiilor de descoperire de locații, dar abordează tocmai aspectele de personalizare și de nuanță pe care platformele existente le tratează insuficient.

2.3 Obiectivele lucrării

Pornind de la contextul și de la limitările prezentate, lucrarea își propune următoarele obiective:

- proiectarea și implementarea unei aplicații mobile native pentru Android, împreună cu un server REST și o bază de date care să susțină stocarea și interogarea datelor despre locații, inclusiv a celor geospațiale;
- dezvoltarea unui algoritm de recomandare personalizată, care să ordoneze locațiile în funcție de potrivirea dintre atributele lor și preferințele utilizatorului;
- realizarea unei componente de descoperire, care să propună locații de calitate pe dimensiuni mai puțin urmărite de utilizator, pentru a evita limitarea la rezultatele deja cunoscute;
- implementarea unei căutări tolerante la greșeli de tastare și insensibile la diacritice, potrivită pentru limba română;

- asigurarea unui mecanism de autentificare sigur și a unei stocări protejate a datelor sensibile pe dispozitiv.

Capitolul 3

Tehnologiile utilizate

Acest capitol prezintă principalele tehnologii folosite în realizarea aplicației și motivează alegerea lor. Nu sunt justificate toate bibliotecile, ci doar deciziile importante pentru proiect; pentru fiecare se arată ce reprezintă tehnologia aleasă, de ce a fost potrivită în acest caz și ce alternative ar fi existat.

O privire de ansamblu asupra stivei tehnologice este prezentată în Tabelul 3.1. Aplicația este formată dintr-un client Android nativ, scris în Kotlin, un server REST realizat cu Spring Boot în Java și o bază de date PostgreSQL găzduită pe platforma Supabase. În continuare sunt discutate deciziile principale.

Nivel	**Tehnologie**
Client mobil	Aplicație Android nativă (Kotlin + Jetpack Compose)
Backend	Spring Boot (Java), API REST, găzduit pe Render
Bază de date	PostgreSQL + PostGIS, găzduită pe Supabase
Sursă de date geospațiale	OpenStreetMap (OSM), extrase cu Overpass Turbo
Servicii externe	Google Places API (New) - căutare text și fotografii
Autentificare	Token-uri JWT, parole protejate cu bcrypt

Tabelul 3.1: Stiva tehnologiilor

3.1 Aplicație nativă Android

Pentru client s-a ales o aplicație nativă Android, scrisă în Kotlin, în locul unei soluții cross-platform precum Flutter sau React Native, care ar fi permis rularea aceluiași cod atât pe Android, cât și pe iOS. Întrucât tema proiectului vizează exclusiv platforma Android, principalul avantaj al soluțiilor cross-platform, acela de a acoperi mai multe sisteme cu un singur cod, nu se aplică. În schimb, o aplicație nativă oferă acces direct la funcțiile platformei, cum ar fi serviciul de localizare sau mecanismele de stocare securizată, fără stratul de abstractizare introdus de framework-urile cross-platform, și permite folosirea directă a bibliotecilor moderne din ecosistemul Android. Din aceste motive, abordarea nativă a fost cea mai potrivită.

3.2 Interfața declarativă cu Jetpack Compose

Interfața aplicației este construită cu Jetpack Compose, un instrument declarativ recomandat oficial de Google, în locul sistemului mai vechi bazat pe fișiere XML. Într-o abordare declarativă, interfața este descrisă ca o funcție a stării aplicației, adică a datelor care reflectă la un moment dat ce trebuie afișat; la modificarea acestor date, framework-ul actualizează automat doar componentele afectate. Spre deosebire de abordarea clasică, în care interfața și starea trebuie sincronizate manual, ceea ce este predispus la erori, Compose se integrează direct cu modelul MVVM utilizat: starea expusă de ViewModel determină ce se afișează. Ca urmare, cantitatea de cod necesară scade, iar interfața rămâne mereu în acord cu datele.

3.3 Serverul cu Spring Boot

Serverul este realizat cu Spring Boot, în Java. Alternativele luate în considerare au fost soluții precum Node.js cu Express, Django sau .NET. Spring Boot a fost ales pentru că oferă un ecosistem matur și componente deja integrate pentru exact nevoile acestui proiect: o abstractizare curată peste baza de date prin Spring Data JPA și Hibernate, suport pentru tipurile geospațiale prin Hibernate Spatial și un mecanism de securitate complet, care include autentificare și protejarea parolelor prin hashing. Față de Node.js cu Express, care permite o prototipare rapidă, dar oferă o structură mai laxă și un suport geospațial mai puțin matur, Spring Boot impune o organizare clară și pune la dispoziție direct aceste funcționalități. La aceasta se adaugă faptul că limbajul Java era deja cunoscut, ceea ce a redus efortul de învățare.

3.4 Autentificarea cu token-uri JWT

Pentru autentificare s-a ales un mecanism stateless, bazat pe token-uri JWT, în locul sesiunilor păstrate pe server. În cazul sesiunilor, serverul ar fi trebuit să rețină starea fiecărui utilizator

conectat, ceea ce complică scalarea. Un token JWT este, în schimb, semnat și autonom: serverul îl poate verifica fără a păstra nicio informație între cereri, ceea ce se potrivește bine atât cu stilul REST, cât și cu un client mobil. Compromisul acestei abordări este că un token nu poate fi revocat ușor înainte de expirare; acesta este atenuat printr-o durată de viață scurtă a token-ului.

3.5 Sursa datelor: OpenStreetMap și Overpass Turbo

Datele despre locații nu au fost introduse manual, ci au fost extrase dintr-o sursă open source, OpenStreetMap [4]. Aceasta este o bază de date geografică colaborativă, care conține informații despre un număr foarte mare de puncte de interes din întreaga lume, precum restaurante, muzee, parcuri sau cafenele, împreună cu poziția lor și cu o serie de detalii asociate. Alegerea OpenStreetMap a fost potrivită din mai multe motive: datele pot fi folosite fără costuri, acoperirea pentru un oraș precum București este bună, iar fiecare locație include deja coordonatele geografice și o clasificare pe categorii, ceea ce se potrivește direct cu modelul de date al aplicației.

Pentru extragerea efectivă a datelor s-a folosit Overpass Turbo, un instrument care permite interogarea bazei OpenStreetMap pe baza unor criterii precise. Cu ajutorul lui au putut fi selectate doar locațiile de interes pentru aplicație, de exemplu cele dintr-o anumită zonă și dintr-un anumit set de categorii, rezultatul fiind apoi exportat și introdus în baza de date proprie. În acest fel, popularea inițială a bazei de date s-a realizat pornind de la date reale, fără a fi nevoie de introducerea manuală a fiecărei locații.

3.6 Google Places API

OpenStreetMap oferă numele, poziția și categoria locațiilor, însă nu conține fotografii și nu acoperă întotdeauna detalii precum adresa exactă sau programul de funcționare. Pentru a completa aceste informații s-a folosit Google Places API, serviciul Google care pune la dispoziție date detaliate despre locații. Acesta a fost ales deoarece oferă o acoperire foarte bună și informații de calitate, iar în combinație cu datele din OpenStreetMap permite obținerea unei prezentări complete a fiecărei locații.

Capitolul 4

Proiectarea și implementarea sistemului

4.1 Arhitectura aplicației

Aplicația are o arhitectură pe trei niveluri (3-tier), un caz particular al modelului N-tier: nivelul de prezentare (clientul Android), nivelul logic (serverul Spring Boot) și nivelul de date (baza de date PostgreSQL). Această împărțire asigură o separare clară a responsabilităților, fiecare nivel ocupându-se de o singură parte a sistemului și comunicând doar cu nivelul vecin prin interfețe bine definite. Clientul nu accesează niciodată direct baza de date, ci trece prin server.

Acest model aduce mai multe avantaje. Nivelurile pot fi dezvoltate și modificate independent: serverul poate fi actualizat fără a reconstrui aplicația mobilă, atâta timp cât contractul API rămâne neschimbat. Baza de date nu este expusă direct clientului, ceea ce reduce riscurile de securitate, iar același server REST ar putea fi folosit ulterior și de alți clienți, de exemplu o aplicație web sau iOS. Tot de aici vine și flexibilitatea în cazul unei migrări viitoare către alte tehnologii, deoarece un nivel poate fi înlocuit fără a afecta restul sistemului.

Clientul trimite cereri HTTPS în format JSON, care includ un token JWT în antet (pentru securitate), către unul dintre endpoint-urile din backend. Backend-ul Spring Boot, găzduit pe internet cu Render, primește cererea și o validează. Pentru accesul la date, Spring folosește Hibernate ca ORM, care traduce operațiile asupra entităților în interogări SQL, trimise apoi către baza de date printr-o conexiune JDBC. Baza de date PostgreSQL, găzduită la rândul ei pe Supabase, conține locațiile importate de pe OpenStreetMap, filtrate și selectate cu ajutorul Overpass Turbo. Când utilizatorul dorește să vizualizeze o locație, acesta intră pe pagina aferentă, care conține mai multe detalii: printre acestea se numără scorul fiecărui atribut individual, adresa, programul de lucru și câteva fotografii de prezentare. Fotografiile, adresa și programul de lucru sunt inițial preluate prin apeluri din backend către Google Places API, cele două din urmă fiind ulterior stocate în baza de date. Figura 4.1 prezintă fluxul client-server.

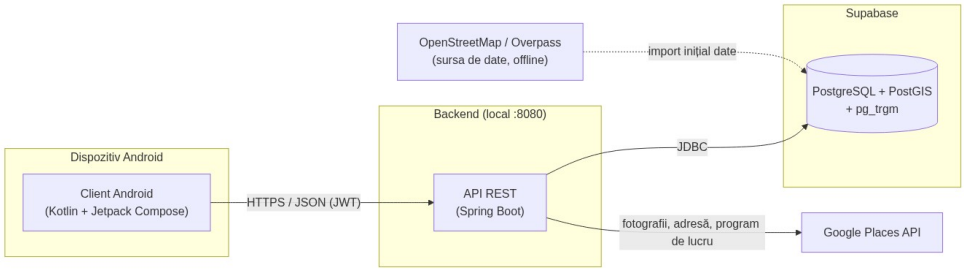


Figura 4.1: Fluxul client-server

4.2 Backend

Backend-ul este o aplicație Spring Boot, împărțită pe cele trei straturi clasice, pentru a obține o separare a responsabilităților: controller, service și repository. Stratul controller expune endpoint-urile REST și primește cererile venite de la client, stratul service conține logica aplicației, iar stratul repository se ocupă de accesul la baza de date. Datele care circulă între aceste straturi sunt reprezentate de entități și obiecte de tip DTO.

La nivel de securitate, Spring Security filtrează cererile către endpoint-urile securizate și verifică, pentru fiecare cerere, semnătura și termenul de expirare ale token-ului JWT; tot Spring Security generează token-ul la autentificare. Întrucât fiecare cerere își poartă propriul token, serverul nu păstrează stare de sesiune între cereri.

La crearea unui cont, parola este trecută prin bcrypt cu un cost de 12 și se stochează doar hash-ul rezultat, împreună cu valoarea aleatoare (salt) folosită, niciodată parola în clar. La autentificare, parola introdusă este trecută prin același algoritm, iar hash-ul este comparat cu cel stocat.

Endpoint-urile sunt organizate pe resurse (locații, recenzii, preferințe) și sunt sintetizate în Tabela 4.1.

Stratul service conține cea mai mare parte din logica aplicației și tot aici se realizează și apelurile către serviciile externe, mai exact preluarea datelor de pe Google Places API. Pentru fotografiile unei locații se verifică mai întâi dacă identificatorii fotografiilor de la Google Places sunt deja salvați pentru acea locație; apelul către Google Places se face doar atunci când aceștia lipsesc. Aplicația nu stochează imaginile propriu-zise, ci doar acești identificatori, pe baza cărora fotografiile sunt încărcate la cerere.

Maparea către baza de date se face la nivelul entităților: sunt mapate doar tabelele și coloanele strict necesare, iar stratul repository se ocupă de accesul efectiv la aceste date. Hibernate rulează cu proprietatea ddl-auto setată pe validate, ceea ce înseamnă că nu modifică niciodată schema, ci doar verifică la pornire că tabelele și coloanele așteptate există. Operațiile de tip DDL se pot face doar din interfața Supabase, printr-un utilizator separat; utilizatorul folosit de aplicația Spring are privilegii limitate strict la operații DML.

Pentru cererile mai complexe, cum sunt cele legate de recomandări, se folosește JOIN FETCH la extragerea atributelor unei locații, pentru a evita problema N+1 – situația în care, pentru

Controller	Endpoint	Rol
Auth	‘POST /auth/register‘·‘/login‘·‘/change-password‘	Cont și sesiune
Location	‘POST /api/locations/nearby‘	Locații apropiate (public)
	‘POST /api/locations/recommended‘	Recomandări personalizate (autentificat)
	‘POST /api/locations/discover‘	”Try something new” (autentificat)
	‘GET /api/locations/search?q=	Căutare ”fuzzy” (public)
	‘GET /api/locations/id/reviews‘	Recenzii locație
	‘GET /api/locations/id/photos‘·‘/photo‘	Fotografii locație
Preference	‘GET‘ / ‘PUT /api/preferences‘	Profilul de preferințe
Review	‘POST /api/reviews‘, ‘GET /api/reviews/me‘	Postare și vizualizare recenzii proprii
Wishlist	‘GET‘ · ‘POST‘ · ‘DELETE /api/wishlist‘	Salvare

Tabelul 4.1: Endpoint-uri REST

o listă de entități, se execută câte o cerere suplimentară pentru fiecare element. Deoarece pentru locații se folosește PostGIS, anumite interogări trebuie scrise ca cereri native, folosind operatori spațiali (de exemplu ST_DWithin) pe care JPQL nu îi poate exprima. O cerere nativă întoarce coloane, dar nu populează automat relațiile dintre entități (graful de atribute al unei locații), motiv pentru care se folosesc două cereri separate: prima extrage doar identificatorii locațiilor, în ordinea corectă, iar a doua încarcă datele complete pe baza acestor identificatori printr-o interogare JPQL, care populează automat entitățile. Același mecanism în doi pași este folosit pentru recomandări, descoperire, căutare și pentru listarea locațiilor din apropiere. Conexiunile la baza de date sunt reutilizate printr-un pool gestionat de HikariCP, pentru a susține mai multe cereri simultane fără a deschide o conexiune nouă la fiecare cerere.

4.3 Client MVVM

În această arhitectură, dispozitivul Android reprezintă strict partea de client (frontend). Această alegere face ca logica aplicației și prelucrarea datelor, aspectele de securitate, autentificare și validare, precum și logica de apelare a serviciilor externe să se afle pe server, în backend. Aplicația Android are mai mult rolul unui strat de prezentare, cu excepția cache-ului local realizat cu Room. Acest cache reduce numărul de apeluri către server și permite afișarea mai rapidă a datelor deja descărcate. În același timp, faptul că logica și datele sensibile rămân pe server este și o decizie de securitate: ele nu sunt incluse în pachetul aplicației (APK), unde ar putea fi inspectate sau decompilate. Tot legat de securitate, token-ul JWT emis de backend pentru validarea cererilor este stocat folosind AndroidX Security Crypto.

Deși, așa cum s-a menționat, partea de Android a aplicației este mai degrabă un strat de prezentare, aceasta folosește o arhitectură de tip Model–View–ViewModel (MVVM), pentru o mai bună organizare a codului sursă. Stratul repository este organizat pe resurse, oglindind controllerele din backend, care la rândul lor sunt împărțite în funcție de endpoint-urile REST, adică pe resursele ce urmează a fi accesate.

Stratul ViewModel asigură comunicarea cu repository-ul; el poate prelucra într-o oarecare măsură datele, pe care apoi le transmite stratului View (Screen). Rolul său principal este de a păstra starea ecranului: ViewModel-ul supraviețuiește schimbărilor de configurație (configuration changes), de exemplu rotirii ecranului, astfel încât datele afișate nu se pierd. Această stare este expusă către View prin clase de tip UiState, pentru o organizare mai clară. Fluxul de date poate fi și invers: datele introduse în View sunt prelucrate în ViewModel și trimise mai departe către repository, pentru a fi salvate sau folosite în cereri către backend. În acest fel, ViewModel-ul are un rol central în asigurarea fluxului de date într-un singur sens (Unidirectional Data Flow) și a separării responsabilităților (Separation of Concerns).

Stratul de prezentare (View) este alcătuit din funcții Compose, care observă starea expusă de ViewModel prin clasele UiState și se recompun automat la modificarea acesteia. Fluxul de date din clientul Android este prezentat în Figura 4.2.

Arhitectura este de tip Single Activity Application, abordarea modernă recomandată împreună cu Compose, deoarece simplifică navigarea și gestionarea stării. Funcțiile Compose corespunzătoare ecranelor aplicației sunt afișate și schimbate dinamic în cadrul aceleiași activități Android. Pentru a permite această navigare și pentru a reține istoricul navigării (back stack) se folosește Jetpack Navigation; tot prin acesta, un utilizator neautentificat este dirijat către ecranul de autentificare.

Deoarece clasele din straturi, în special cele din stratul ViewModel, depind de alte clase, pentru a reduce codul necesar instanțierii și transmiterii acestor dependențe și pentru o organizare și o testare mai ușoară, se folosește un instrument de dependency injection, și anume Hilt.

Spre deosebire de o aplicație web tradițională, o aplicație Android trebuie să funcționeze, măcar parțial, și atunci când dispozitivul este offline, întrucât utilizatorul se poate afla în mișcare. Din acest motiv, datele despre locații sunt stocate într-o bază de date locală SQLite, care are rolul de cache local, gestionată cu ajutorul bibliotecii Room. La fiecare cerere se preiau de la backend ceva mai multe date decât cele afișate imediat, astfel încât, atunci când utilizatorul cere mai multe (de exemplu apăsând ”View more”), acestea să poată fi afișate instant și să rămână disponibile chiar dacă, între timp, conexiunea s-a pierdut.

Pentru o comunicare eficientă cu serverul, se folosește biblioteca Retrofit, construită peste biblioteca OkHttp, care se ocupă de formatarea și de comunicarea HTTPS cu serverul. Tokenul JWT este adăugat automat la fiecare cerere, atunci când este nevoie, printr-un interceptor OkHttp. Retrofit adaugă un nivel de abstractizare care permite ca dezvoltarea să se concentreze pe funcționalitățile aplicației, fără gestionarea directă a straturilor și protocoalelor de comunicare în rețea.

Pentru un aspect vizual unitar și bine organizat în toată aplicația, se folosesc teme, culori și componente UI definite global, precum și un font preluat din Downloadable Google Fonts. Pentru un ecran nou sau o funcționalitate nouă, în majoritatea cazurilor se pot refolosi în mare parte componentele deja existente.

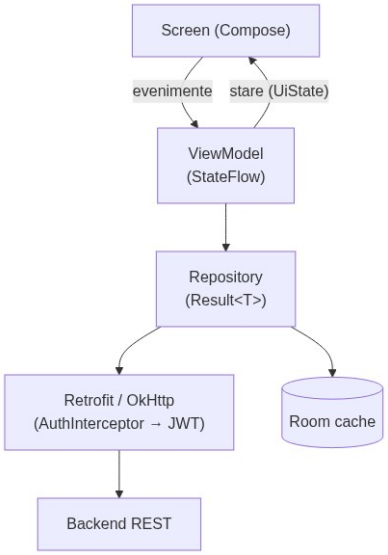


Figura 4.2: Fluxul clientului Android

4.4 Baza de date

Baza de date este de tip PostgreSQL și este găzduită pe Supabase, o platformă de gestionare care păstrează accesul complet la SQL și la extensiile PostgreSQL, preluând în același timp partea de găzduire. Pentru stocarea și interogarea eficientă a coordonatelor geospațiale se folosește extensia PostGIS [9]: coordonatele unei locații sunt reținute ca tip geography(Point, 4326), iar căutările pe rază se realizează cu funcția ST_DWithin, care calculează distanțele în metri. Pentru căutarea tolerantă la greșeli după numele locațiilor se folosește extensia pg_trgm, care măsoară similaritatea dintre șiruri pe baza trigramelor (secțiunea 2.1.4). Modelul de date complet este prezentat în diagrama entitate-relație din Figura 4.3.

În aplicație există 10 tipuri de locații, iar pentru acestea sunt definite în total 83 de atribute. La prima vedere pare a exista o redundanță, de exemplu, existența unui atribut ”atmosferă” pentru restaurant și un atribut ”atmosferă” pentru muzeu. Acestea sunt însă atribute separate în mod intenționat: deși se referă la aceeași caracteristică, importanța și modul în care se manifestă diferă în funcție de tipul de locație, iar fiecare atribut are și o pondere proprie (global_weight) folosită la calcularea scorului de potrivire. Pentru ca utilizatorul să nu fie expus celor 83 de atribute brute, acestea au fost grupate în 23 de ”concepte” afișate în interfață, de unde provine și necesitatea tabelei attribute_concept.

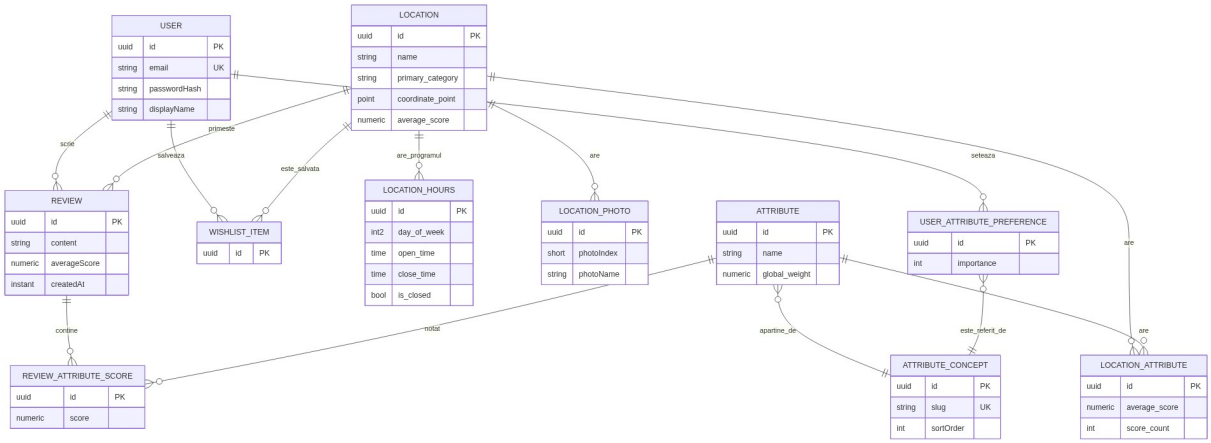


Figura 4.3: Diagrama ER

4.5 Algoritmii folosiți

Nucleul aplicației îl reprezintă modul în care locațiile din apropiere sunt ordonate în funcție de cât de bine se potrivesc cu gusturile utilizatorului. La baza acestei ordonări stau trei calcule înrudite: scorul de potrivire, care generează recomandările personalizate, scorul de descoperire, folosit pentru sugestiile de tip ”Try something new”, și clasarea rezultatelor căutării tolerante la greșeli. Toate pornesc de la scorurile medii ale atributelor, motiv pentru care acestea vor fi descrise mai întâi.

4.5.1 Scorurile medii ale atributelor

Schema bazei de date respectă a treia formă normală, cu excepția scorurilor medii, care sunt denormalizate intenționat din motive de performanță.

Așa cum a fost descris în modelul de date, fiecărei locații îi corespunde un set de atribute specifice categoriei sale, iar fiecare atribut primește scoruri în recenziile utilizatorilor. În loc ca media unui atribut să fie recalculată din toate recenziile la fiecare citire, ceea ce ar fi costisitor, aceasta este păstrată precalculată: pentru fiecare pereche (locație, atribut) se reține scorul mediu și numărul de scoruri primite. La sosirea unei recenzii noi, media este actualizată pornind de la media veche, numărul vechi de scoruri și noua valoare, fără a mai parcurge recenziile anterioare. La fel se actualizează și punctajul general al locației. Această abordare este o formă de denormalizare: se acceptă un mic efort suplimentar la scriere și o anumită redundanță, în schimbul unor citiri rapide, ceea ce este de dorit într-o aplicație în care datele sunt citite mult mai des decât scrise.

Scorurile astfel obținute, notate în continuare S_{la} (scorul mediu al atributului a la locația l), reprezintă intrarea pentru algoritmii descriși mai jos.

4.5.2 Scorul de potrivire

Scorul de potrivire măsoară cât de bine corespund atributele unei locații preferințelor exprimate de utilizator. Acesta combină două ponderări diferite. Prima este una personală: pentru fiecare dintre cele 23 de concepte, utilizatorul a stabilit o importanță pe o scară de la 0 la 5, unde 0 înseamnă că respectivul aspect nu îl interesează. A doua este una structurală: fiecare atribut are o pondere globală proprie, W_a, care reflectă cât de mult ar trebui să conteze acel atribut indiferent de utilizator. Deoarece utilizatorul își exprimă preferințele la nivel de concept, iar locațiile sunt evaluate la nivel de atribut, importanța unui concept se transmite tuturor atributelor asociate lui.

Pentru o locație l, scorul se calculează ca raport între două sume ponderate, luate peste atributele a ale locației:

$$\text{potrivire}(l) = \frac{\sum_a S_{la} \cdot P_{c(a)} \cdot W_a}{\sum_a P_{c(a)} \cdot W_a} \tag{4.1}$$

unde $P_{c(a)}$ este importanța acordată de utilizator conceptului $c(a)$ căruia îi aparține atributul a. La sumă contribuie doar conceptele cu importanță mai mare decât 0. Împărțirea la suma ponderilor transformă rezultatul într-o medie ponderată a scorurilor pe atribute, cuprinsă între 0,5 și 5; aceasta este apoi adusă pe scara de la 0 la 100 și afișată utilizatorului ca procent de potrivire. Folosirea unei medii ponderate, și nu a unei simple sume, permite ca locații cu un număr diferit de atribute evaluate să fie comparate corect între ele.

În situația în care utilizatorul nu are preferințe relevante, fie pentru că nu este autentificat, fie pentru că a lăsat toate importanțele pe 0, numărătorul și numitorul se anulează, iar scorul nu mai are sens. În acest caz, aplicația revine la ordonarea după punctajul general al locațiilor, astfel încât lista de recomandări rămâne utilă chiar și înainte de stabilirea preferințelor. Un sistem de recomandare ar trebui să se comporte rezonabil în lipsa datelor, nu să întoarcă o listă goală [12].

4.5.3 Scorul de descoperire

Recomandarea strict personalizată are un dezavantaj cunoscut: utilizatorul ajunge să vadă tot timpul doar variații ale lucrurilor care îi plac deja, fenomen numit uneori ”bulă de filtrare” [10]. Funcționalitatea de descoperire urmărește exact opusul: scoaterea în evidență a unor locații bune, dar pe dimensiunile pe care utilizatorul le consideră de obicei puțin importante.

Pentru aceasta se folosește aceeași formulă ca la scorul de potrivire, însă importanța fiecărui concept este înlocuită cu o anti-importanță, definită ca

$$antiP_c = 5 - P_c. \tag{4.2}$$

Astfel, un concept căruia utilizatorul i-a acordat importanța 0 capătă ponderea maximă 5 și invers. Reutilizând întregul mecanism, dar cu vectorul de preferințe inversat, se obțin locații care în mod normal nu s-ar potrivi gusturilor sale. Pentru ca rezultatul să rămână valoros, se

aplică în plus un prag de calitate: sunt luate în considerare doar locațiile al căror punctaj general depășește aproximativ 3,5. Acest prag face diferența între o sugestie utilă, o locație bună care altfel nu ar fi fost aleasă, și una pur întâmplătoare. Dacă utilizatorul nu are preferințe definite, inversul acestora nu are sens, iar lista de descoperire rămâne goală, secțiunea nefiind afișată.

4.5.4 Căutarea tolerantă la greșeli și clasarea rezultatelor

Căutarea manuală funcționează și atunci când utilizatorul greșește la tastare: în loc de o potrivire exactă, care nu ar întoarce niciun rezultat pentru ”muzue” în loc de ”muzeu”, se folosește potrivirea pe baza similarității de trigrame (secțiunea 2.1.4).

Atunci când mai multe locații se potrivesc, ordinea în care sunt afișate este importantă. Rezultatele sunt ordonate astfel: mai întâi cele al căror nume începe chiar cu textul căutat, apoi cele cu cea mai mare similaritate de trigrame, atât pe nume, cât și pe categorie, iar la egalitate se folosește punctajul general ca departajare. Această ordine corespunde așteptărilor unui utilizator: ceea ce a scris efectiv apare primul, iar potrivirile apropiate urmează.

Pentru ca această căutare să nu fie afectată de diacritice sau de litere mari și mici, atât numele stocate, cât și textul introdus de utilizator trec prin aceeași prelucrare. Numele este normalizat prin descompunere Unicode (forma NFD) [13], în care o literă precum ”ț” este separată într-o literă de bază ”t” și un semn diacritic; eliminând semnele diacritice se obține forma simplă, fără diacritice. Forma normalizată este calculată o singură dată, la salvarea locației, și este folosită ulterior la fiecare căutare.

Capitolul 5

Prezentarea aplicației

Acest capitol prezintă aplicația din perspectiva utilizatorului, parcurgând principalele funcționalități în ordinea în care acestea ar fi întâlnite în mod firesc. Pentru fiecare funcționalitate se descrie ce poate face utilizatorul, ce vede pe ecran și, pe scurt, ce se întâmplă la nivelul implementării.

Trebuie menționat de la început că interfața aplicației este în limba engleză; prin urmare,

în text vor fi citate ca atare etichetele afișate pe ecran (de exemplu ”Recommended for you”), însoțite, acolo unde este util, de traducerea lor.

5.1 Pornirea aplicației și autentificarea

La pornire, aplicația se deschide direct pe ecranul principal, fără a obliga utilizatorul să se autentifice. Astfel, o persoană poate explora locațiile încă de la prima deschidere, autentificarea fiind cerută doar atunci când dorește să folosească o funcție care presupune un cont, cum ar fi salvarea unei locații sau scrierea unei recenzii. Tot la pornire, aplicația cere permisiunea de acces la locația dispozitivului, necesară pentru a afișa locațiile din apropiere.

Atunci când utilizatorul alege să se autentifice, ajunge pe ecranul de autentificare (”Welcome back”), unde introduce adresa de e-mail și parola, parola putând fi afișată sau ascunsă. Dacă nu are încă un cont, poate trece la ecranul de înregistrare (”Create account”), unde își alege un e-mail, un nume de utilizator și o parolă, confirmând-o. Pe acest ecran, aplicația verifică în timp real câteva condiții simple, de exemplu faptul că parola are cel puțin opt caractere, indicația afișată actualizându-se pe măsură ce utilizatorul tastează. La nivelul serverului, în urma unei autentificări reușite, este generat un token, stocat apoi în siguranță pe dispozitiv și folosit ulterior pentru a identifica utilizatorul.

5.2 Stabilirea preferințelor

Imediat după crearea contului, utilizatorul este îndrumat către un scurt proces de configurare a preferințelor (Figura 5.1). Acesta este realizat sub forma unui asistent cu câte o întrebare pe ecran, în care utilizatorul stabilește importanța a șase aspecte esențiale: atmosfera, animația, calitatea mâncării, raportul calitate-preț, curățenia și calitatea serviciului. Pentru fiecare, întrebarea este de forma ”How much does … matter to you?”, iar importanța se alege pe o scară de la 0 la 5, fie dintr-un selector cu butoanele ”–” și ”+”, fie atingând o bară împărțită în cinci segmente. Sub aceasta, o etichetă descrie valoarea aleasă, de la ”Don’t care” (valoarea 0) până la ”Essential” (valoarea 5). La final, utilizatorul poate începe explorarea sau poate alege să își ajusteze în detaliu profilul.

S-a optat pentru un set redus de șase aspecte în această etapă tocmai pentru a păstra procesul scurt și simplu, fără a expune utilizatorul, chiar de la început, întregului set de concepte disponibile. Acestea din urmă, în total 23, pot fi configurate ulterior dintr-un ecran dedicat de preferințe, accesibil din profil. Acolo, conceptele sunt grupate pe categorii care pot fi extinse sau restrânse, fiecare având aceeași scară de importanță de la 0 la 5, iar un indicator arată câte dintre ele au fost setate.

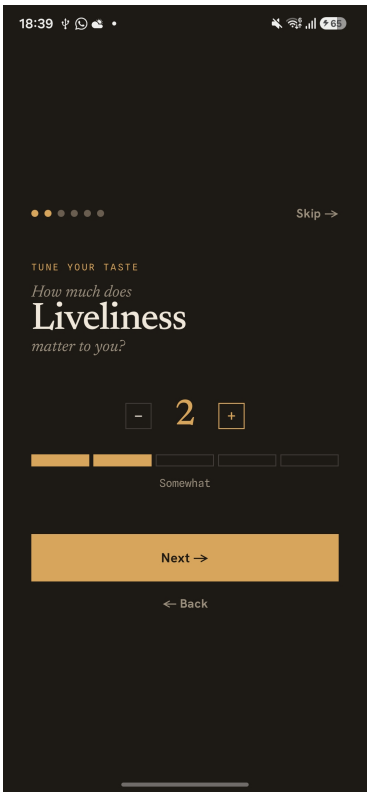


Figura 5.1: Asistentul de configurare al preferințelor

5.3 Ecranul principal și descoperirea locațiilor

Ecranul principal reprezintă punctul central al aplicației și conține locațiile din apropierea utilizatorului, organizate în până la trei liste. Prima, ”Recommended for you” (recomandate pentru tine), conține locațiile ordonate după scorul de potrivire cu preferințele utilizatorului, fiecare locație afișând procentul de potrivire (Figura 5.2). A doua, ”Try something new” (încearcă ceva nou), propune o singură locație de descoperire, pe care utilizatorul o poate schimba apăsând un buton de reîncercare, pentru a primi o altă sugestie. A treia listă, ”Top rated nearby” (cele mai bine cotate din apropiere), conține locațiile ordonate după punctajul lor general și este afișată întotdeauna.

Primele două liste apar doar atunci când utilizatorul este autentificat și și-a stabilit preferințele; în lipsa acestora, ecranul afișează numai lista publică, ordonată după punctaj. În acest fel, aplicația rămâne utilă și pentru un utilizator neautentificat, oferind în același timp recomandări personalizate celui care și-a exprimat preferințele. Fiecare locație este prezentată sub forma unui card cu o fotografie, categoria și numele locației, punctajul general și, în cazul listei personalizate, procentul de potrivire. Inițial sunt afișate cinci locații, iar restul pot fi încărcate treptat printr-un buton ”View more”.

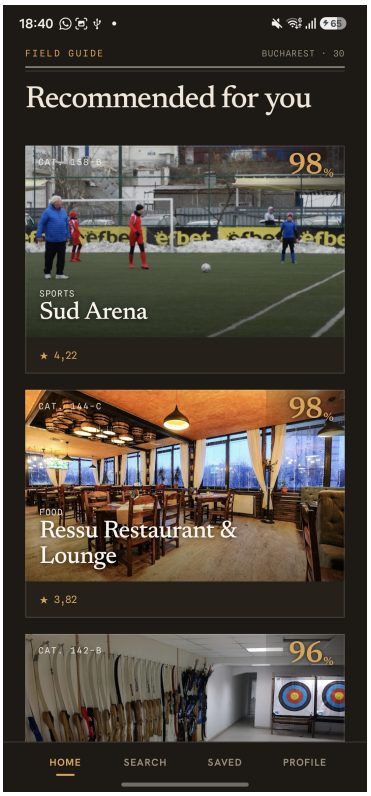


Figura 5.2: Ecranul principal cu recomandările

5.4 Căutarea locațiilor

Pe lângă descoperirea pe ecranul principal, utilizatorul poate căuta o locație după nume sau categorie. Căutarea funcționează pe măsură ce utilizatorul tastează: sub câmpul de căutare apare o listă de sugestii, fiecare conținând numele și categoria unei locații (Figura 5.3). Apăsând pe o sugestie, utilizatorul ajunge direct la pagina locației respective. Dacă în schimb confirmă căutarea, sugestiile dispar și sunt afișate rezultatele complete, sub forma acelorași carduri de locație.

La nivel de implementare, pentru a evita trimiterea unei cereri la fiecare tastă apăsată, aplicația așteaptă o scurtă pauză în tastare înainte de a efectua căutarea. Căutarea este tolerantă la greșeli de tastare, astfel încât o mică eroare nu împiedică găsirea locației dorite, mecanismul fiind cel descris în capitolul dedicat algoritmilor.

5.5 Pagina unei locații

Atunci când utilizatorul deschide o locație, ajunge pe o pagină care conține toate informațiile despre aceasta, organizate de sus în jos. În partea de sus se află o galerie de fotografii ce pot fi parcurse, peste care sunt afișate categoria, punctajul general și numele locației. Urmează scorul de potrivire, afișat ca procent (”Match for you”); dacă utilizatorul nu și-a stabilit preferințele, în loc de procent este afișat un mesaj care îl invită să facă acest lucru.

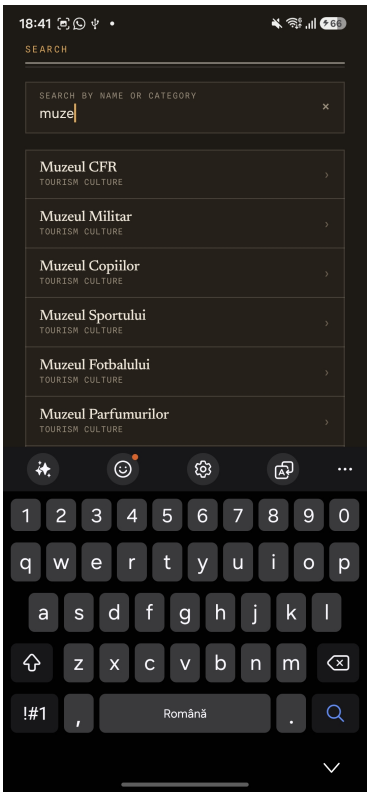


Figura 5.3: Ecranul de căutare

Sub scorul de potrivire se află un tabel al atributelor locației, în care, pentru fiecare atribut, sunt afișate importanța acordată de utilizator și scorul mediu obținut din recenzii. Mai jos sunt prezentate, atunci când sunt disponibile, adresa locației, informații despre accesibilitate și grupuri sanitare, precum și programul de funcționare, însoțit de un indicator care arată dacă locația este deschisă în acel moment. În continuare se află secțiunea de recenzii, unde utilizatorul poate citi părerile celorlalți și de unde poate începe scrierea unei recenzii proprii. În partea de jos a paginii se găsesc butonul de salvare a locației și cel de afișare a indicațiilor de orientare, care deschide aplicația de hărți a dispozitivului.

5.6 Scrierea unei recenzii

Un utilizator autentificat poate lăsa o recenzie pentru o locație. Pe ecranul de recenzie, acesta acordă un scor atributelor locației, fiecare atribut fiind notat pe o scară de cinci stele, cu posibilitatea de a acorda și jumătăți de stea. Sub atribute se află un câmp în care utilizatorul poate scrie un text liber. Pentru ca recenzia să poată fi trimisă, utilizatorul trebuie să acorde un scor pentru cel puțin trei atribute; în caz contrar, este afișat un mesaj corespunzător, iar trimiterea este blocată. După trimitere, scorurile acordate contribuie la actualizarea mediilor atributelor locației, conform mecanismului descris anterior.

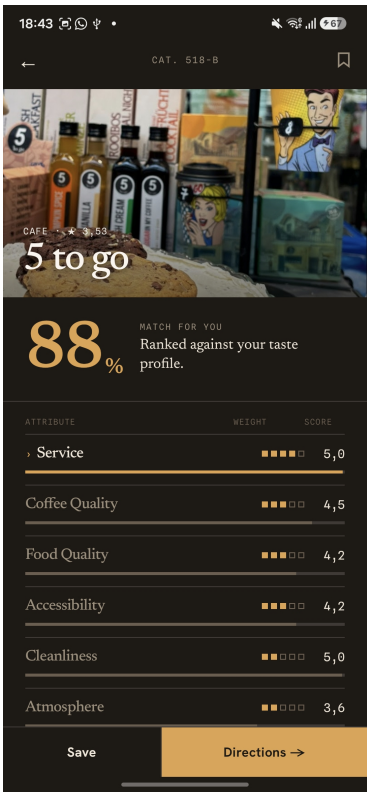


Figura 5.4: Pagina unei locații

5.7 Salvarea locațiilor și profilul

Utilizatorul poate salva orice locație care îl interesează, atingând simbolul corespunzător de pe pagina locației sau de pe cardul acesteia. Locațiile salvate sunt adunate pe un ecran dedicat, de unde pot fi accesate din nou sau eliminate. Dacă utilizatorul nu este autentificat, acest ecran îl invită să se autentifice.

În cadrul profilului, utilizatorul își poate vedea recenziile scrise, fiecare cu locația, scorul acordat și textul recenziei. Tot de aici poate accesa editorul complet de preferințe sau se poate deconecta. În acest fel, profilul adună la un loc activitatea utilizatorului și opțiunile legate de contul său.

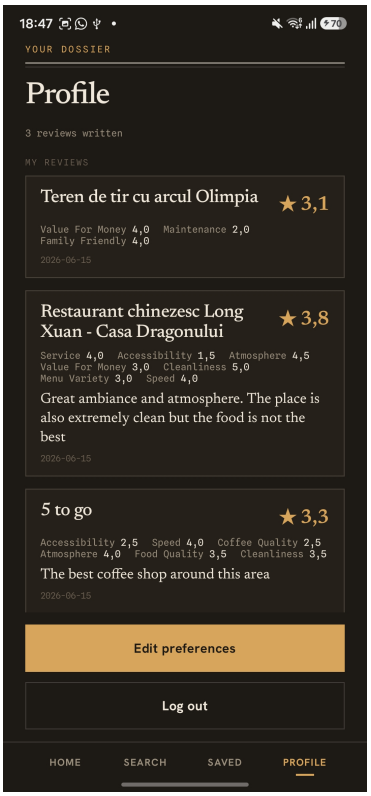


Figura 5.5: Pagina profilului

Capitolul 6

Concluzii

În această lucrare am proiectat și am implementat FindOut, o aplicație mobilă pentru Android care ajută utilizatorii să descopere locații dintr-un oraș în funcție de preferințele lor personale. Sistemul realizat este format dintr-un client Android nativ, un server REST și o bază de date relațională cu suport geospațial, fiecare dintre obiectivele propuse la începutul lucrării fiind atins. Am dezvoltat un algoritm de recomandare personalizată, care ordonează locațiile după potrivirea dintre atributele lor și preferințele utilizatorului, precum și o componentă de descoperire, care propune locații bune pe dimensiuni mai puțin urmărite de acesta. De asemenea, am implementat o căutare tolerantă la greșeli de tastare și insensibilă la diacritice, potrivită pentru limba română, și un mecanism de autentificare bazat pe token-uri, împreună cu o stocare protejată a

datelor sensibile pe dispozitiv.

Datele despre locații sunt limitate la orașul București, de unde au fost extrase, astfel încât aplicația nu poate fi încă folosită în alte orașe. În plus, aplicația este disponibilă doar pe Android, chiar dacă serverul REST ar putea fi folosit și de alți clienți.

Pornind de la aceste limitări, există mai multe direcții de dezvoltare ulterioară. Una dintre ele ar fi extinderea setului de date și către alte orașe. La nivelul algoritmului, recomandarea nu ține în acest moment cont de popularitatea unei locații; o posibilă îmbunătățire ar fi introducerea unei influențe reduse a popularității, astfel încât locațiile foarte puțin cunoscute să nu fie favorizate exclusiv pe baza scorului de potrivire. Nu în ultimul rând, întrucât logica aplicației se află pe server, aceasta ar putea fi extinsă cu un client pentru iOS sau pentru web, precum și cu funcționalități noi.

Consider că rezultatele obținute sunt relevante, deoarece arată că recomandarea de locații poate fi tratată într-un mod personalizat, care ține cont de gusturile fiecărui utilizator, spre deosebire de abordarea uniformă a platformelor existente. Pe lângă utilitatea pentru utilizatori, o astfel de aplicație poate avea și un efect practic asupra economiei locale, îndreptând atenția și către locații mai noi sau mai puțin cunoscute, nu doar către cele deja populare. Abordarea bazată pe preferințe exprimate pe mai multe atribute nu este specifică doar domeniului turistic și ar putea fi adaptată și pentru recomandarea altor tipuri de produse sau servicii.

Bibliografie

[1] Android Developers, *Guide to app architecture*, Google, URL: https://developer.android.com/topic/architecture (accesat în 15.6.2026).

[2] Roy Thomas Fielding, „Architectural Styles and the Design of Network-based Software Architectures”, Teză de doct., University of California, Irvine, 2000, URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm (accesat în 15.6.2026).

[3] Antonin Guttman, „R-trees: A Dynamic Index Structure for Spatial Searching”, în *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, New York, NY, USA: Association for Computing Machinery, 1984, pp. 47–57, DOI: 10.1145/602259.602266.

[4] Mordechai Haklay și Patrick Weber, „OpenStreetMap: User-Generated Street Maps”, în *IEEE Pervasive Computing* 7.4 (2008), pp. 12–18, DOI: 10.1109/MPRV.2008.80.

[5] Michael B. Jones, John Bradley și Nat Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015, DOI: 10.17487/RFC7519, URL: https://www.rfc-editor.org/rfc/rfc7519 (accesat în 15.6.2026).

[6] Steve Krug, *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*, a 3-a ed., Berkeley, CA, USA: New Riders, 2014.

[7] Pasquale Lops, Marco de Gemmis și Giovanni Semeraro, „Content-based Recommender Systems: State of the Art and Trends”, în *Recommender Systems Handbook*, ed. de Francesco Ricci, Lior Rokach, Bracha Shapira și Paul B. Kantor, Boston, MA, USA: Springer, 2011, pp. 73–105, DOI: 10.1007/978-0-387-85820-3_3.

[8] Gonzalo Navarro, „A Guided Tour to Approximate String Matching”, în *ACM Computing Surveys* 33.1 (2001), pp. 31–88, DOI: 10.1145/375360.375365.

[9] Regina O. Obe și Leo S. Hsu, *PostGIS in Action*, a 3-a ed., Shelter Island, NY, USA: Manning Publications, 2021.

[10] Eli Pariser, *The Filter Bubble: What the Internet Is Hiding from You*, New York, NY, USA: Penguin Press, 2011.

[11] Niels Provos și David Mazières, „A Future-Adaptable Password Scheme”, în *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*, USENIX Association, 1999, pp. 81–91.

[12] Francesco Ricci, Lior Rokach și Bracha Shapira, ed., *Recommender Systems Handbook*, a 2-a ed., New York, NY, USA: Springer, 2015, DOI: 10.1007/978-1-4899-7637-6.

[13] The Unicode Consortium, *Unicode Standard Annex #15: Unicode Normalization Forms*, ed. de Mark Davis și Ken Whistler, 2024, URL: https://www.unicode.org/reports/tr15/ (accesat în 15.6.2026).

[14] Esko Ukkonen, „Approximate String-Matching with q-grams and Maximal Matches”, în *Theoretical Computer Science* 92.1 (1992), pp. 191–211, DOI: 10.1016/0304-3975(92)90143-4.